Cursed_Ai Training Handbook

How to teach your own models, offline: language models, speech recognition for your accent, voices, and office documents

Written for this PC (RTX 5060 laptop GPU with 8 GB, Windows) and for the Cursed_Ai app as it stands after the Fine-tune and Train Speech pages were added.

> **How to use this handbook**
> Read **Part 1** first (10 minutes). Then follow the recipe that matches your goal in **Part 3**. Part 4 lists what to do while you still have internet. Part 6 is for when something goes wrong.

Everything in this handbook that was measured on this PC is marked **measured**. Anything not tried yet is marked **untested**.

Contents

1. The big picture and the words you need
2. Where everything lives on this PC
3. Recipes: subject expert, office letters, Indian-accent speech recognition, a voice, image LoRA
4. Before you go offline: the download checklist
5. Step-by-step: training a language model in the app
6. Troubleshooting
7. Settings, times and sizes
8. Quick reference: formats, prompts, commands
9. Glossary

1. The big picture and the words you need

Training here means **fine-tuning**: you start from a model that already works and nudge it with your own examples. You never train from nothing; that would need huge data and many GPUs.

What you train	What it learns	Page in the app
Language model (LLM)	A subject, a writing style, or a document format, from question-and-answer examples	FineTune LLM
Speech to text	To hear your accent, vocabulary and recording conditions (Whisper)	Train Speech, mode "Speech to text"
Text to speech	To speak in one voice or language (XTTS-v2) (untested)	Train Speech, mode "Text to speech"
Image LoRA	A character or look for Flux image generation	Train LoRA

Words you will meet

Base model	The model you start from. Bigger knows more but trains slower and needs more memory.
Dataset	Your examples. For a language model: a file where each line is one instruction and its ideal answer. For speech: audio clips with a text file of what is said.
Adapter / LoRA	A small add-on that holds what was learned. The base model is not changed; the adapter is applied on top. This is why training fits an 8 GB card.
Epoch	One full pass over all your examples. 3 to 8 is typical.
Loss	A score of how wrong the model is on your examples. It should fall (for example from 2.5 to about 1). Lower is not always better: too low means it memorised your examples.
WER / CER	Word (or character) error rate for speech. 0% is perfect. The page scores clips the model never trained on, before and after.
GGUF	One compressed file Ollama can run. Training produces an adapter; the app merges it into the model and converts it to GGUF so it shows up in your Ollama list.
4-bit vs full precision	A 4-bit model is small but cannot be merged and exported. A full-precision model can. Training always loads the model in 4-bit by itself, so you only need to download the full-precision one.

What fine-tuning is good at, and what it is not

Fine-tuning is good at **style, format, habits and vocabulary**: how your office writes a letter, a tone, a layout, an accent. It is only fair at **exact facts** (numbers, signal patterns, article numbers): a small model needs each fact repeated many times in different wording, and may still get some wrong. **Measured:** the IRPCS test model learned the rules' wording but still gave wrong answers on two facts that were in its training data. For facts, keep the source documents and check the answer against them.

2. Where everything lives on this PC

Folder (under D:\Claude\In_Progress\02_Web_Apps\Cursed_Ai)	What is in it
`train\llm_finetune\models`	Base language models. Present now: **Phi-3.5-mini-instruct** (full precision, 7.6 GB), Phi-3.5-mini-instruct-bnb-4bit (cannot export), **Qwen2.5-0.5B-Instruct** (full precision, 1 GB)
`train\llm_finetune\datasets`	Your language-model datasets (`.jsonl`), for example irpcs_combined_v2.jsonl
`train\llm_finetune\outputs\<run>`	Each training run: `adapter\` and, if exported, `gguf\` with the GGUF file and its Modelfile
`train\llm_finetune\env`	The Python environment that does the training (do not move or edit it)
`train\speech\datasets\<name>`	Speech clips with transcripts: clip01.wav and clip01.txt side by side
`train\speech\models`	Whisper models (whisper-tiny is present) and, later, XTTS-v2
`train\speech\outputs\<run>`	Speech results: `model\` (speech to text), `final\` (a trained voice)
`train\speech\tts`	The separate Python environment for voice training (made by the Set up button)
`C:\pinokio\api\fluxgym.git`	FluxGym, used by the Train LoRA page (images)

Starting the app: double-click the Cursed_Ai shortcut or `Cursed_Ai.bat` ; it opens at `http://localhost:3000` . **Do not close or restart the app while a training job is running**: it stops the job.

3. Recipes

Recipe A: a subject expert (for example collision regulations, IRPCS)

1. Put your reference PDFs or Word files into **FineTune LLM → Dataset → Build from your material**. Choose a local writer model such as `qwen2.5:7b` (works offline).
2. Let it write pairs, then **read and fix them**. Delete pairs about court cases, tables, or anything a person would never ask.
3. Add **hand-written pairs for the facts that must be exact**, each asked three or four different ways, with a short direct answer (see "How to write good pairs", Part 5).
4. Train (settings in Part 5), export to Ollama, then ask the model questions it was not trained on.
5. If answers are wrong: add more wordings of the missing facts, raise epochs by 2, then retrain. Repeat.

What to expect (measured, IRPCS on Phi-3.5-mini)

182 pairs, 3 epochs: wording learned, many facts wrong. 519 pairs, rank 32, 5 epochs: better (head-on rule, sternlight arc and flash rate right) but the port-turn signal and the lights of a vessel not under command were still wrong. More wordings and more epochs help; a larger base model helps more.

Recipe B: an office assistant for letters, applications, faxes and emails

Goal: a local model that writes in *your* office's format and language. **Base model:** Qwen2.5-3B-Instruct (strong in Chinese, Arabic and many languages; good English) or Phi-3.5-mini (English). If your office writes in Hindi, try Qwen2.5-3B first and compare. Download it while online (Part 4).

1. **Collect 200 to 500 real documents** (letters, leave and transfer applications, circulars, fax cover sheets, emails). Use final, correct versions only.
2. **Remove private details**: replace names, phone numbers, addresses, account numbers with plausible dummies ("Mr. A. Sharma", "[Ref. No.]"). The model learns the pattern, not the person.
3. **For each document write the request that would produce it**, as a person would type it: "Leave application to the Principal for 3 days for a family function." The document itself is the answer. The in-app pair writer makes questions about facts, so for letters make these pairs yourself or with a chat assistant (a ready prompt is in Part 8).
4. **Cover the variety**: at least 10 to 20 examples of each document type, in the languages you really use, and a few examples that include the details (dates, names) in the request so the model learns to use them.
5. **Put the layout in the answer exactly as you want it**: letterhead lines, "To", subject line, salutation, closing, enclosures. Whatever is consistent in your examples becomes the model's habit.
6. Train 4 to 6 epochs at rank 16 to 32. Export to Ollama. Test with 10 requests you did not train on.
7. **Keep an exact template in the prompt for strict parts** (reference-number format, official wording). Fine-tuning gives style; a pasted template gives exactness.

Start small

Train first on 50 documents of one type (say applications). If the layout comes out right, add the other types. It is far quicker to fix a small dataset than a large one.

Recipe C: speech recognition that understands Indian-accented English

The default speech models are weakest on accents and vocabulary they heard little of. Fine-tuning Whisper on recordings of Indian speakers is exactly the fix. Choose **language: English** (not Hindi) when the speech is English with an Indian accent.

1. **Record the clips.** Several different speakers (at least 5 to 10), the same kind of speech you want recognised (office talk, dictation, meetings), a quiet room, the same microphone you will use later. Each clip **3 to 15 seconds**, never over 30 seconds. wav, mp3, m4a, flac, ogg all work.
2. **How much:** about 30 minutes is enough to see a change; 2 to 5 hours gives a solid improvement. Quality matters more than quantity.
3. In **Train Speech → Clips**: create a dataset, add the recordings, then press **Write draft transcripts** (use whisper-small or bigger for better drafts) or import a sheet you already have.

4. **Correct every transcript** so it is exactly what was said, including names and local words. This is the most important step; wrong text teaches wrong spellings.
5. Download **whisper-small** (about 970 MB, online) if you have not. **Train**: method LoRA, rank 32, epochs 5, learning rate 1e-3, batch 8 (lower to 4 if memory runs out), keep 10% back for scoring.
6. Read the result: the log ends with `RESULT WER 18.0% -> 11.0%` (example). If it got worse, use fewer epochs (3), more clips, or a lower learning rate (5e-4).
7. **Try it:** record yourself, run the base model, then your trained model on the same recording.

> **Important limit**
> The app's microphone button currently uses a very small English-only Whisper (whisper-tiny.en). A model you train on this page is **not yet connected to that button**; use the Try it step to check it. Connecting a trained model to the microphone needs a format conversion that has not been built. A cheaper first step is to switch the microphone button to a larger Whisper such as small.

Measured: the whole chain (draft transcripts, LoRA and full training, before-and-after scoring, Try it) ran correctly on whisper-tiny with 20 English clips. That test only proves the process works; it says nothing about accuracy on Indian accents.

Recipe D: a voice (text to speech) (untested)

The page trains **XTTS-v2**, which can speak Hindi, Arabic and Chinese. It is **licensed for non-commercial use only**. Even without any training, XTTS-v2 can copy a voice from a short reference clip; training makes the copy more faithful and the pronunciation better.

1. Press **Set up** (about 4 GB, online) and **Download XTTS-v2** (about 2 GB, online). The code for this was written but **not yet run**; expect to fix things on the first run.
2. Record **one speaker**, clean audio, no music or echo, **30 minutes or more**, in clips of 1 to 11 seconds, each with an exact transcript.
3. Press **Prepare for voice training** (converts to 22 kHz, drops clips outside 1 to 11 s, picks a reference clip).
4. Train: 6 epochs, batch 2, accumulation 8, learning rate 5e-6. Keep the batch at 1 or 2 on 8 GB.
5. Try it: choose the voice, type a sentence in that language, press Say it.

Recipe E: image LoRA (Train LoRA page)

1. Dataset tab: create a folder, add 15 to 40 varied pictures of the character, write a made-up **trigger word** (for example `richie01`), and caption the pictures (the page can write captions).
2. Train tab: choose the Flux base model and VRAM preset (8G), start. The page's own **Guide** button explains each step with pictures.
3. Copy the finished `.safetensors` to ComfyUI's loras folder with the page's button, then use the trigger word in prompts.

4. Before you go offline: the download checklist

Works offline: all training, merging and exporting to GGUF, adding to Ollama, writing pairs with local Ollama models, draft transcripts with a downloaded Whisper, trying results. **Needs internet:** downloading models, the one-time voice setup, cloud models (the ":cloud" ones and Gemini), and Hugging Face downloads of any kind.

> **Test once before you travel**
> Turn Wi-Fi off and run a very short training (1 epoch on a handful of examples, no export). Training and exporting should work offline, because the models and tools are all on this PC and the GGUF tool (llama.cpp) was already installed during the first export, but this has not been tried with the network off.

Download	Size	Status on this PC
unsloth/Phi-3.5-mini-instruct (language model)	7.6 GB	Downloaded
unsloth/Qwen2.5-0.5B-Instruct (small, fast)	1.0 GB	Downloaded
unsloth/Qwen2.5-3B-Instruct (multilingual, for the office assistant)	6.2 GB	Not downloaded
unsloth/Llama-3.2-3B-Instruct (alternative)	6.4 GB	Not downloaded
unsloth/gemma-2-2b-it (lightest 2B)	5.2 GB	Not downloaded
openai/whisper-tiny	0.15 GB	Downloaded
openai/whisper-small (recommended for accents)	1.0 GB	**Not downloaded: get this first**
openai/whisper-medium (best, needs LoRA)	3 GB	Not downloaded
Voice setup (Python environment)	about 4 GB	Not set up
coqui/XTTS-v2 (voice model)	about 2 GB	Not downloaded

Use the page's **Download** buttons ("Need a base model?" on FineTune LLM, "Need a Whisper model?" on Train Speech). They keep what was already downloaded if the connection drops, so just press the button again. Keep at least **25 GB free** on drive D: for a language-model export.

> **Also bring your material**
> Copy your reference PDFs and documents, recordings and any ChatGPT-made datasets onto this PC before you go offline, so you only need the app.

5. Step by step: training a language model in the app

Step 1: choose the base model

Model	Good for	Notes
Qwen2.5-0.5B-Instruct	Quick experiments, very narrow tasks	Fast and small; holds little, makes more mistakes (not yet tested on your data)
Phi-3.5-mini (3.8B)	English subject experts, letters	Safest fit for 8 GB. Used for the IRPCS tests
Qwen2.5-3B	Office assistant, Hindi, Arabic, Chinese	Best multilingual choice among these
Llama-3.2-3B, gemma-2-2b	Alternatives	Try if the others disappoint

Always use the **full-precision** download (names without "bnb-4bit") so the GGUF export works.

Step 2: build the dataset

Open **FineTune LLM → 1 Dataset**. There are three ways in, and you can mix them:

- **Build from your material:** add PDF, Word, text, pictures or web links; a writer model makes question-and-answer pairs. Press **Write pairs**.
- **Write examples by hand:** type instruction, optional input, expected output.
- **Import:** a `.jsonl`, `.json` or `.csv` file made elsewhere (for example in ChatGPT). Columns accepted: instruction / input / output, prompt / completion, question / answer, or chat "messages".

In **Your dataset** every row is editable: instruction, input and output. The two small buttons on a row (redo, add) write a new pair from the same passage. The line under a row says where it came from. When happy, type a name and press **Save as .jsonl**. **Your work is kept if you refresh the page**, but save it anyway.

How to write good pairs

- **Ask each fact several ways.** "What sound signal means a turn to port?", "How do I signal altering to port by whistle?", "What does two short blasts mean?" all get the same answer. This teaches the fact, not one sentence.
- **Short, direct answers** for facts ("Two short blasts."), full documents for letters.
- **Never contradict yourself.** Two pairs with different answers to the same question teach confusion. Search the file for the key words after you write.
- **Delete junk:** pairs that say "according to the text", questions nobody would ask, tables copied as text.
- **Test questions are not training questions.** Keep a separate list of 10 questions you never train on; they tell you the truth.

Your goal	Pairs to aim for
One narrow task (one letter type)	50 to 150
A subject with many facts	500 to 2,000, several wordings per fact

An office assistant (many document types)	200 to 500 real documents

Step 3: set the training

Go to **2 Fine-tune**. Pick the base model and your saved dataset. Settings (the (i) button on each card explains them):

Setting	Starting value	Change it when
LoRA rank	16 (32 for many facts)	Raise to 32 or 64 if the model will not learn facts; costs more memory
LoRA alpha	same as rank	Rarely
Epochs	3 (5 for small datasets)	Raise by 2 if answers are vague; lower if it repeats training text or ignores other questions
Learning rate	2e-4	Lower (1e-4) if answers get worse or the loss jumps around
Batch size	2	Set 1 if you run out of memory
Max length	1024 (512 for short pairs)	Raise only for long documents; longer means more memory
Export a GGUF	on	Off for a quick try; on to use the model in Ollama
Ollama model name	for example irpcs-v1	Use a new name for every version

Press **Start fine-tune**. The band under the title shows the stage and a percent bar; the log at the bottom can be enlarged with the expand button. A popup tells you when it is finished or failed, on any tab.

Step 4: export and use it

1. With export on, after training the app merges the adapter into the model, converts it to GGUF, and registers it in Ollama under your name (**measured:** about 6 to 12 extra minutes for Phi-3.5-mini).
2. The model then appears in `ollama list` and in the app's model list. Run it with `ollama run your-name`.
3. **It was trained on a fixed prompt layout** ("### Instruction: ... ### Response:"). When the export registers the model itself (the "Ollama model name" setting), it sets this layout and a context of 4096 for Ollama. **Use that setting rather than the separate "Add to Ollama" button on the Finished models card**, which still writes a plain Modelfile without the layout. If you load the GGUF in another program (LM Studio, llama.cpp) you must set the same template, or answers get worse (Part 8).
4. **Keep the adapter and your dataset.** To improve the model later, add examples to the dataset and train again from the base model; do not stack training on top of an exported model, because each round tends to blur what the last one learned.

Step 5: test and improve

1. Ask 10 questions that were **not** in the training set, and 3 unrelated ones ("What is the capital of France?") to confirm it still behaves normally.
2. Note which answers are wrong. For each, check the dataset: missing? contradicted? asked only one way?
3. Add several wordings of each missing fact, retrain, and compare with the previous version by keeping both models in Ollama under different names.

Topics: to teach a second topic, add its pairs to the same dataset and retrain, including some pairs from the first topic. Training only on the new topic tends to make the model forget the old one.

6. Troubleshooting

What you see	What it means and what to do
"Out of memory" or the run stops early	Batch size 1, max length 512, close Ollama models, ComfyUI and other GPU programs, or pick a smaller model. For speech use a smaller batch or LoRA.
Export fails: "base model should be a 16bit ... base model"	You trained from a 4-bit model. Download the full-precision version and train from it. After the app is restarted it blocks this choice before training starts.
Download stuck at the same number of MB	Press Stop, then Download again; it resumes. Or download the file yourself in your browser or downloader into the model's folder with exactly the same file names.
Download error "getaddrinfo failed"	Your internet dropped. Reconnect and press Download again.
Ollama: "model requires more system memory (45 GiB)"	The model's context window is too large. A model registered through the "Ollama model name" setting already has `num_ctx 4096`. For an older one, add `PARAMETER num_ctx 4096` to its Modelfile and run `ollama create name -f Modelfile` again.
The job disappeared or the log is empty	The app was restarted, which stops running jobs. Start it again, and do not restart during training.
Answers are wrong or vague	More wordings per fact, 2 more epochs, higher rank (32), a bigger base model, and a check for contradicting pairs.
It repeats training text, or gets odd on other questions	Too many epochs or too high a learning rate. Lower both, or add varied examples.
Not enough disk space during export	The export briefly needs about 3 times the model size (merged copy, intermediate file, final file). Free at least 25 GB for a 3.8B model.
Speech: "ffmpeg was not found"	ffmpeg must be on PATH (it is on this PC). Restart the app after installing it.
Speech: draft transcripts are poor	Use whisper-small or medium for the drafts, and correct them by hand.
Speech error rate got worse after training	Fewer epochs, lower learning rate, more and more varied clips, check the transcripts for mistakes.
Voice setup says Python 3.10 to 3.12 is needed	Voice training does not support newer Python. Install Python 3.10, 3.11 or 3.12 and press Set up again.

7. Settings, times and sizes

Times measured on this PC (RTX 5060 laptop, 8 GB)

Run	Time
Phi-3.5-mini, 182 pairs, 3 epochs, rank 16 (training only)	about 3 minutes

Phi-3.5-mini, 519 pairs, 5 epochs, rank 32 (training)	about 9 minutes
GGUF export and registering in Ollama (Phi-3.5-mini)	about 6 to 12 minutes more
Whisper-tiny, 20 clips (test of the whole chain)	under a minute plus model loading

As a rough guide, Phi-3.5-mini takes **0.2 to 0.35 seconds per example per pass**: pairs times epochs times that. A 0.5B model should be several times faster, but this has not been measured. Writing pairs with a local 7B model took about 15 minutes for 40 passages.

Space

Phi-3.5-mini full-precision base	7.6 GB
Exported GGUF of Phi-3.5-mini	2.3 GB each version
Adapter and checkpoints per run	a few hundred MB

Delete old runs you do not need (the run folders in `outputs`) and old Ollama models with `ollama rm name`.

8. Quick reference

Dataset file format (language model)

One JSON object per line, UTF-8, extension `.jsonl`:

```
{"instruction": "Write a leave application to the Principal for 3 days.", "input": "", "output":
"To,\nThe Principal,\n...\nYours faithfully,\n..."}
{"instruction": "What sound signal means I am altering course to port?", "input": "", "output":
"Two short blasts."}
```

Line breaks inside a text are written as `\n`. Hindi, Arabic and Chinese text can be typed as is.

A prompt to make training pairs from your real documents with a chat assistant

Paste a document after this prompt, remove private details first, and import the answer with **Import .jsonl**:

```
I am building a training file for a small local model that writes office documents.
Below is a real document. Reply with ONE line of JSON Lines, nothing else, in this shape:
{"instruction": "...", "input": "", "output": "..."}
Rules:
- "instruction" is the short request a clerk would type to get this document
  (who it is to, what it is about, key details such as dates and amounts).
- "output" is the document exactly as given, with the same layout and line breaks (use \n).
- Keep the language of the document. Do not add or change anything.
- Replace any real name, phone number or account number with a dummy.

DOCUMENT:
<paste the document here>
```

For many documents, ask for one line per document and paste several at once, separated by a line of dashes.

Prompt layout the exported models expect

```
### Instruction:
<your question>

### Response:
```

If you use a trained GGUF in another program, set this as the prompt template, and add "### Instruction:" as a stop word. The app does this for you inside Ollama.

Useful commands (Windows Command Prompt or PowerShell)

```
ollama list                     (show your models)
ollama run irpcs-colregs-v2     (chat with a model; type /bye to leave)
ollama rm name                  (delete a model)
ollama create name -f Modelfile  (register a GGUF again after editing its Modelfile)
```

Speech clips: layout on disk

```
train\speech\datasets\hindi_office\
    clip01.wav     clip01.txt    (text file holds exactly what is said)
    clip02.mp3     clip02.txt
    ...
```

A sheet to import transcripts can be lines like `clip01.wav|the words spoken` or a Common Voice `.tsv` (columns `path` and `sentence`).

Checklist: one training session

- ☐ Base model downloaded (full precision) and enough free disk (25 GB or more)
- ☐ Dataset saved; contradicting and junk pairs removed; test questions kept apart
- ☐ Ollama models and ComfyUI closed if memory is tight
- ☐ Settings checked; unique name for this version
- ☐ App left running and untouched until the "finished" popup
- ☐ 10 unseen questions asked; results noted; next changes decided

9. Glossary

Accumulation	How many small batches are combined before one learning step. Lets a small GPU act like a bigger batch.
Checkpoint	A saved copy of training at the end of an epoch.
Context window	How much text a model can read at once. Bigger uses more memory in Ollama.
Hugging Face	The website the models are downloaded from.
Overfitting	The model memorised the examples and does worse on anything new.
Quantisation (q4_k_m)	Shrinking a model to about a quarter of its size with little quality loss. The export uses q4_k_m.
Trigger word	A made-up word that calls a trained character up in image prompts.
Whisper	The speech-recognition model family fine-tuned on the Train Speech page.
XTTS-v2	The text-to-speech model family (Hindi, Arabic, Chinese and more) used for voices. Non-commercial licence.